

CSE331

AUTOMATA AND COMPUTABILITY

Reductions and the halting problem

Summer 2026

Today's learning goals

Sipser Ch 5.1

- Define and explain core examples of decision problems:
 A_{DFA} , E_{DFA} , EQ_{DFA} , A_{TM} , **$HALT_{\text{TM}}$**
- Define reductions from one problem to another.
- Use reductions to prove undecidability.

$A_{\text{TM}} \rightarrow \text{solved}$

$HALT_{\text{TM}} \rightarrow ??$

So far

Decidable	Recognizable (and not decidable)	Co-recognizable (and not decidable)
✓ A_{DFA}	✓ A_{TM}	✓ A_{TM}^C
$L(D) = \{ \}$ ✓ E_{DFA}		
EQ_{DFA}		
✓ $INFINITE_{DFA}$		
Rev_{DFA}		

Give algorithm!

Diagonalization

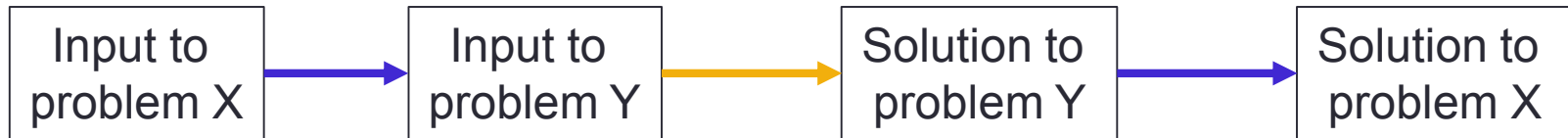
Reductions

Sipser p. 216

- Goal: solve $\text{problem } X$
- Assume we have a way to solve other $\text{problem } Y$
- Reduction: **translation** between X and Y

Reductions

- Goal: solve **problem X**
- Assume we have a way to solve other **problem Y**
- Reduction: **translation** between X and Y



Reductions

mean solving problem X using a solver for problem Y.

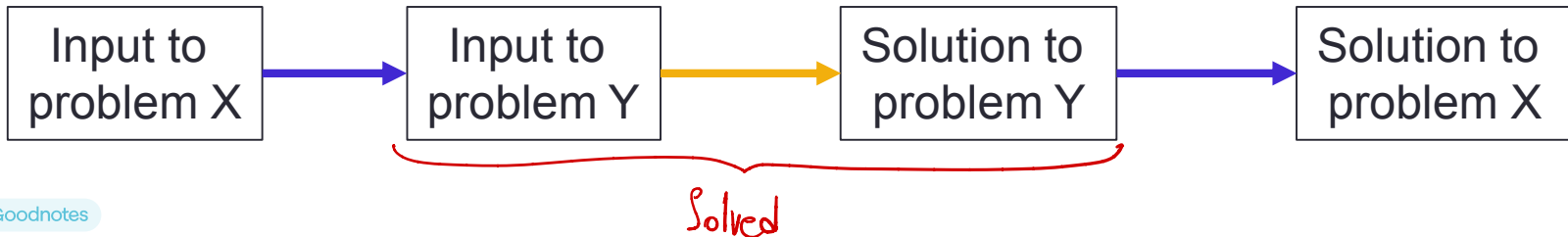
- Goal: solve **problem X**
- Assume we have a way to solve other **problem Y**
- Reduction: **translation** between X and Y

$X \leq Y$, means Y is at least as hard as X.

Translate:
input to X
to input to Y

Solve Y

Translate:
solution to Y
to solution to X



GCD is the largest number that divides both number exactly.

LCM is the smallest positive number that both numbers divide exactly

For numbers 12 and 18

$$\text{gcd}(12, 18)$$

Factors is a number that divides another number exactly

12 : 1, 2, 3, 4, 6, 12

18 : 1, 2, 3, 6, 9, 18

The Common factors are : 1, 2, 3 and 6

$$\text{gcd}(12, 18) = 6$$

List their factors and find the greatest factor.

$$\text{LCM}(12, 18)$$

A multiple is obtained by multiplying a number by 1, 2, 3 - —

Multiple of 12 : 12, 24, 36, 48 - —

Multiple of 18 : 18, 36, 54, 72 - —

The first Common multiple is 36.

$$\text{LCM}(12, 18) = 36$$

$$\text{lcm} \leq \text{gcd}$$

Goal : Solve Problem X

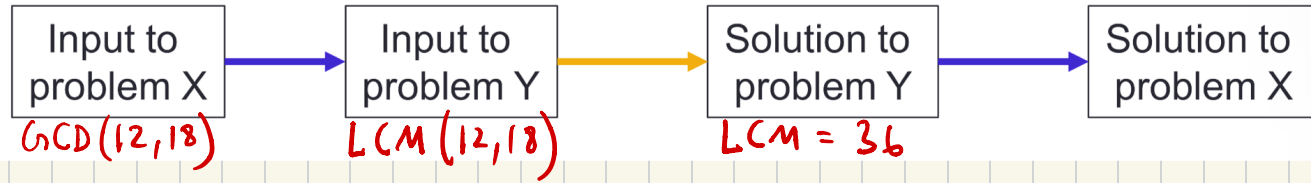
Problem X : Find the GCD (the greatest Common factor)

Problem Y : Find the LCM (the Smallest Common multiple)

Translate:
input to X
to input to Y

Solve Y

Translate:
solution to Y
to solution to X



We use the LCM Solver and translate its answer using:

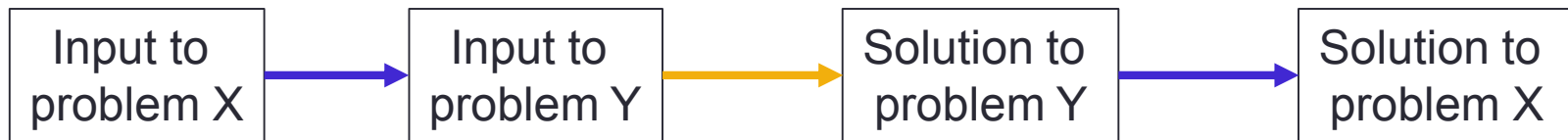
$$gcd(a, b) = \frac{a \times b}{LCM(a, b)}$$

$$gcd(12, 18) = \frac{12 \times 18}{LCM(12, 18)} = \frac{12 \times 18}{36} = 6$$

This is called reducing gcd to LCM.

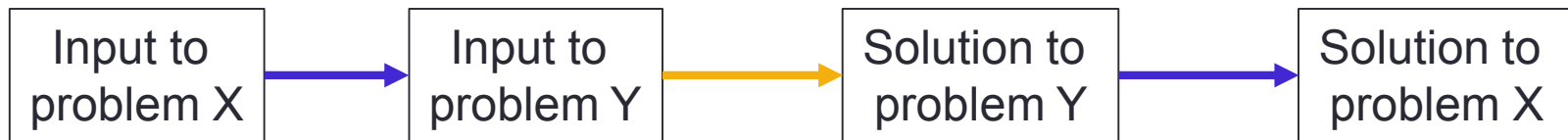
Reductions

- Goal: solve **problem X**
- Assume we have a way to solve other **problem Y**
- Reduction: **translation** between X and Y
- Consequence: if Y is “**easy**”, then X is “**easy**” (**decidable**)



Reductions

- Goal: solve **problem X**
- Assume we have a way to solve other **problem Y**
- Reduction: **translation** between X and Y
- Consequence: if X is “**hard**”, then Y is “**hard**” (**undecidable**)



The halting problem!

$\text{HALT}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ halts on input } w \}$

$A_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$

*[Reject] $w \notin L(M)$
[Accept] $w \in L(M)$*

How is HALT_{TM} related to A_{TM} ?

- A. They're the same set.
- B. HALT_{TM} is a subset of A_{TM}
- C. A_{TM} is a subset of HALT_{TM}
- D. They have the same type of elements but no other relation.
- E. I don't know.

$$\text{HALT}_{TM} = \{ \langle M, w \rangle \mid M \text{ eventually stops on } w \}$$

Stop means that M either accepts or rejects. It just must not run forever.

What M does on w	In A_{TM} ?	In HALT_{TM} ?
Accept	Yes	Yes
Rejects	No	Yes
Loop forever	No	No

→ Will it stop?

→ Will it stop specifically by accepting?

Therefore, every pair in A_{TM} is also in HALT_{TM} , but not vice versa.

$$A_{TM} \subsetneq \text{HALT}_{TM}$$

→ Will it stop?

→ Will it stop specifically by accepting

The halting problem!

$\text{HALT}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ halts on input } w \}$

$A_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$

↓
Undecidable

The reduction proof will justify this
We cannot conclude merely from

$A_{\text{TM}} \subsetneq \text{HALT}_{\text{TM}}$

Which of these sets is harder?

- A. HALT_{TM}
- B. A_{TM}
- C. They're both decidable
- ☒ D. They're both undecidable.
- E. I don't know.

Claim: HALT_{TM} is undecidable.

- Proof by contradiction ...

Imaginary machine, assume its a decider.

M 's behaviour	What G says	Answer to "Does M accept"
Accepts	Halt	Run M , see accept.
Rejects	Halt	Run M , see reject
Runs forever	Does not halt	Immediately answer No.

HALT_{TM} Reject A_{TM}

Claim: HALT_{TM} is undecidable.

- Proof by contradiction ...

$$A_{\text{TM}} \subsetneq \text{HALT}_{\text{TM}}$$



- Assume we have a machine G that **decides** HALT_{TM}
- Build an algorithm **that uses G as a subroutine** to decides A_{TM}
- But this is impossible!
- So G cannot possibly exist, hence HALT_{TM} is undecidable.

Note: Sipser uses R for the machine G . (p. 216)

Claim: HALT_{TM} is undecidable.

$\rightarrow$ Imaginary Halting decider

- Let G be the TM that is assumed to decide HALT_{TM}



- Construct a new TM M_{ATM} by: "On input $\langle M, w \rangle$ "

1. Run G on input $\langle M, w \rangle$. M_{ATM} ask G : "Will M stop on w "

2. If G rejects, then reject. $\rightarrow$ M runs forever. M never accept, so M_{ATM} reject.

3. If G accepts, run M on w .

a. If this computation accepts, accept.

b. If this computation rejects, reject."

Accept Reject Run forever

① A_{TM} asks: Will the result be Accept?

② HALT_{TM} asks: Will the program finish at all!

Assume G can answer the second question.

So using G , we could always determine M accepts w . That would decide A_{TM} , which is impossible. Therefore G cannot exist and HALT_{TM} is undecidable

Claim: HALT_{TM} is undecidable.



- Let G be the TM that is assumed to decide HALT_{TM}
- Construct a new TM M_{ATM} by: "On input $\langle M, w \rangle$ "
 1. Run G on input $\langle M, w \rangle$.
 2. If G rejects, then reject.
 3. If G accepts, run M on w .
 - a. If this computation accepts, accept.
 - b. If this computation rejects, reject."

G accepts $\Rightarrow M$ will eventually stop

G accepts $\neq M$ accept
 $w \in L(M) \} \text{Halts.}$
 $w \notin L(M) \}$
 $w \in L(M)$

Which of the machines are deciders?

- A. All of them: G , M_{ATM} , M
- ☒ B. Definitely G and M_{ATM} ; M may or may not be.
- C. Definitely G ; M_{ATM} and M may or may not be.
- D. None of them has to be.
- E. I don't know.

Claim: HALT_{TM} is undecidable.

- Let G be the TM that is assumed to decide HALT_{TM}
- Construct a new TM M_{ATM} by: "On input $\langle M, w \rangle$ "
 1. Run G on input $\langle M, w \rangle$.
 2. If G rejects, then reject.
 3. If G accepts, run M on w .
 - a. If this computation accepts, accept.
 - b. If this computation rejects, reject."



Lemma: M_{ATM} decides A_{TM} .

Proof of correctness of construction....

"Does the new machine M_{ATM} work correctly in every case?"

What $M(w)$ does	What G Says	What M_{ATM} does
Accepts.	"Yes, it halts"	Run M and accepts
Rejects	"Yes it halts"	Run M and rejects
Run forever	"No, does not halt"	Reject immediately.

$HALT_{TM}$ is Undecidable.

This would decide A_{TM} - But A_{TM} is undecidable.

∴ Our assumption that G exist must be false.

In Case 2: G accepts even when M rejects, because G only checks whether M halts. Both accepting + rejecting, count as halting.

Claim: HALT_{TM} is undecidable.

- **Lemma:** M_{ATM} decides A_{TM}
- Proof: there are 3 cases
 1. **$M(w)$ accepts:**
 - $G(\langle M, w \rangle)$ accepts
 - $M(w)$ accepts $\rightarrow M_{\text{ATM}}$ accepts
 2. **$M(w)$ rejects:**
 - $G(\langle M, w \rangle)$ accepts
 - $M(w)$ rejects $\rightarrow M_{\text{ATM}}$ rejects
 3. **$M(w)$ runs forever:**
 - $G(\langle M, w \rangle)$ rejects $\rightarrow M_{\text{ATM}}$ rejects

Assume:
 G decides HALT_{TM}

$M_{\text{ATM}}(\langle M, w \rangle)$:

1. Run G on input $\langle M, w \rangle$
2. If G rejects, then reject
3. If G accepts, run M on w
 - a. If this computation accepts, accept.
 - b. If this computation rejects, reject."

Claim: HALT_{TM} is undecidable.



- Let G be the TM that is assumed to decide HALT_{TM}
- Construct a new TM M_{ATM} using G

$$A_{\text{TM}} \not\subseteq \text{HALT}_{\text{TM}}$$

Problem $X : A_{\text{TM}}$, Problem $Y : \text{HALT}_{\text{TM}}$

- **Lemma:** M_{ATM} decides A_{TM}

Assumed solver for J : machine G .

$$A_{\text{TM}} \leq \text{HALT}_{\text{TM}}$$

- Since we know A_{TM} is undecidable, HALT_{TM} is undecidable
- Proof using reduction!

Reduction

$$A_{TM} \leq HALT_{TM}$$

"Problem X **reduces to** problem Y" means $X \leq Y$

"Problem X is no harder than problem Y" means

"Given an algorithm deciding problem Y, we could solve problem X" means

"Given a solution for Y, we have a solution for X"

$$A_{TM} \leq \{aa\}$$

Which is **not** true?

- A. A_{TM} reduces to $HALT_{TM}$
- B. EQ_{DFA} reduces to E_{DFA}
- C. Σ^* reduces to A_{TM}
- D. E_{DFA} reduces to $\{\}$ (the empty set)
- ☒ E. A_{TM} reduces to $\{aa\}$

E_{DFA} = DFA does not accept any string.